

CS486 SOLUTION - TEST 2

Based on the FINAL REVIEW structure; total: 10 points

Question 1 - ER Models

[1 pt]

Solution. Department(DeptCode) has global key DeptCode. Doctor is identified within Department, so a natural composite key is (DeptCode,EmpNo). Patient has key PID. Appointment connects exactly one Doctor and one Patient; cardinality Doctor 1:N Appointment and Patient 1:N Appointment. The constraint that one doctor cannot have two appointments at the same time yields a candidate key (DeptCode,EmpNo,ApptDateTime). If a surrogate AID is used, add UNIQUE(DeptCode,EmpNo,ApptDateTime). Prescription is optional 0:1 from Appointment and can use AID as both PK and FK.

Question 2 - Query Languages

[2 pt]

Solution. (a) Use a self-join of Appointment with renamed copies:

$$\pi_{DID}(\sigma_{a_1.DID=a_2.DID \wedge a_1.PID \neq a_2.PID}(\rho_{a_1}(Appointment) \times \rho_{a_2}(Appointment))).$$

(b)

$$\{p \mid Patient(p) \wedge \neg \exists a (Appointment(a) \wedge a.PID = p.PID \wedge a.Fee > 500)\}.$$

(c)

```
WITH rev AS (
  SELECT d.DID,d.Name,d.Dept,
         COALESCE(SUM(CASE WHEN a.ApptDate >= '2026-01-01'
                           AND a.ApptDate < '2027-01-01'
                           THEN a.Fee ELSE 0 END),0) AS revenue
  FROM Doctor d LEFT JOIN Appointment a ON a.DID=d.DID
  GROUP BY d.DID,d.Name,d.Dept
), x AS (
  SELECT *, DENSE_RANK() OVER(PARTITION BY Dept ORDER BY revenue DESC) rk
  FROM rev
)
SELECT Dept,DID,Name,revenue FROM x WHERE rk=1;
```

This naturally keeps zero-revenue doctors only when zero is the departmental maximum.

Question 3 - Database Constraints

[1 pt]

Solution. A foreign key Prescription(AID) REFERENCES Appointment(AID) enforces appointment existence. The department authorization rule can be stated as no forbidden tuple:

$$\neg \exists pr \exists a \exists d (Prescription(pr) \wedge Appointment(a) \wedge Doctor(d) \wedge pr.AID = a.AID \wedge a.DID = d.DID \wedge \neg \exists g (DrugAuthor$$

Implement with an INSERT/UPDATE trigger on Prescription that joins inserted rows to Appointment and Doctor and rejects any row lacking a matching DrugAuthorization(Drug,Dept).

Question 4 - Serialization & Concurrency**[2 pt]**

Solution. Edges: X gives $T_1 \rightarrow T_2$, $T_1 \rightarrow T_3$, and $T_2 \rightarrow T_3$; Y gives $T_2 \rightarrow T_3$ and $T_2 \rightarrow T_1$ (because $w_2(Y)$ precedes $r_1(Y)$). Thus $T_1 \leftrightarrow T_2$ is a cycle, so not conflict-serializable.

$r_2(X)$ reads T_1 's uncommitted write; $r_3(Y)$ reads T_2 's uncommitted write. Commit order c_1, c_2, c_3 respects those read-from dependencies, so the schedule is recoverable. It is not cascadeless because the reads occur before the writers commit.

For the lock-order example, wait-for graph has $T_1 \rightarrow T_2$ (waiting for Y) and $T_2 \rightarrow T_1$ (waiting for X), a deadlock cycle. Resolve by aborting one victim, e.g. the younger/cheaper transaction, releasing its locks, and restarting it later.

Question 5 - Recovery**[1 pt]**

Solution. Deferred update means uncommitted updates were not installed in the database, so there is nothing to undo. T_1 and T_3 committed and are winners; redo their logged updates as necessary: $A := 15$ and $C := 31$. T_2 never committed, so ignore its B and D records. No UNDO is required.

Question 6 - Functional Dependencies & Normalization**[1 pt]**

Solution. AB is a key: $AB \rightarrow C \rightarrow D \rightarrow A$ and $A \rightarrow E$, giving all attributes. BC is also a key because $C \rightarrow D \rightarrow A$, then with B obtain all. BD is a key because $D \rightarrow A \rightarrow E$ and AB gives C. Minimal candidate keys include AB, BC, and BD.

$C \rightarrow D$ has C not a superkey and D is prime (D occurs in candidate key BD), so it does not violate 3NF; $A \rightarrow E$ has A not a superkey and E nonprime, so it violates 3NF. With composite keys, A is a proper subset of AB and determines nonprime E, so 2NF is violated. Highest NF is 1NF.

For $R_1(A, C, D, E)$ and $R_2(A, B)$, intersection is A. In R_1 , $A \rightarrow E$ only, so A does not determine all of R_1 ; in R_2 , A does not determine B. Therefore the binary lossless-join criterion is not satisfied; not guaranteed lossless.

Question 7 - Concurrency Control Execution**[1 pt]**

Solution. Example: T_2 reads $B=1000$, writes 1050; before T_2 aborts, T_1 reads 1050, writes 950, and commits; then T_2 aborts. T_1 performed a dirty read. Because T_1 committed based on an uncommitted value that was rolled back, cascading recovery is required: T_1 must also be undone (or the schedule is unrecoverable if T_1 already committed before T_2 abort/commit status was known). Strict/cascadeless scheduling prevents this.

Question 8 - Query Optimization**[1 pt]**

Solution. (a) PID equality via hash: about 2 index/bucket I/Os plus a data-page fetch (roughly 3); full scan = 10,000. Hash clearly wins.

(b) Five percent of rows = 25,000 rows = about 500 clustered data pages. Cost $\approx 4 + 500 = 504$ page I/Os versus 10,000 for a scan, so clustered B+ tree wins. A hash index is unsuitable for an ordered date range because hashing destroys key order; it supports equality well but not contiguous range traversal.

– End of Solutions –